

Exercise #1 - Invariant Schedules for LCM of all Request Periods

Mason McGaffin

Exercise #1

ECEN 5623 Real-Time Embedded Systems

University of Colorado Boulder

June 05, 2026

Contents

1 RATE MONOTONIC POLICY	3
2 APOLLO 11 LUNAR LANDER.....	5
2.1 SUMMARY.....	5
2.2 ROOT CAUSE ANALYSIS	5
2.3 RATE MONOTONIC SCHEDULING ANALYSIS	6
2.4 APPLICATION TO THE APOLLO 11 MISSION.....	7
3 RT-CLOCK	7
3.1 SOURCE CODE DESCRIPTION	7
3.2 WHICH CLOCK IS BEST?	9
3.4 DESIGN TARGETS.....	10
4 SCHEDULING USING LINUX PTHREADS.....	11
4.1 SIMPLETHREAD.....	11
4.2 INCDECTHREAD	15
4.3 DEADLINE – SCHED_DEADLINE AND SCHED_FIFO	18
4.4 SEQUENCER.....	19
5 REFERENCES.....	22
6 APPENDIX	23

Note. For the RTES course, I, Mason McGaffin, will be using the Jetson Orin Nano as my embedded system of choice. Although this system is over resourced for our application, it is convenient to use given my current situation.

1 Rate Monotonic Policy

The Rate Monotonic Policy states that services which share a CPU core should multiplex it (with context switches that preempt and dispatch tasks) based on priority, where highest priority is assigned to the most frequently requested service and lowest priority is assigned to the least frequently requested AND total shared CPU core utilization must preserve some margin (not be fully utilized or overloaded).

a) Draw a timing diagram for three services S1, S2, and S3 with T1=3, C1=1, T2=5, C2=2, T3=15, C3=3 where all times are in milliseconds. [Note that you can find examples of timing diagrams here – note that we have not yet covered dynamic priorities, just RM fixed policy described here, so ignore EDF and LLF for now].

b) Label your diagram carefully and describe whether you think the schedule is feasible (mathematically repeatable as an invariant indefinitely) and safe (unlikely to ever miss a deadline).

c) What is the total CPU utilization by the three services?

The core principle of rate monotonic policy is that tasks with a shorter period have a higher priority. Therefore, S₁ (T₁=3) > S₂ (T₂=5) > S₃ (T₃=15). S₁ has the highest priority

Next, we must analyze the schedulability. First, we find utilization.

$$\begin{aligned}
 U &= \sum_{i=1}^n \frac{C_i}{T_i} \\
 &= \frac{1}{3} + \frac{2}{5} + \frac{3}{15} \\
 &= \frac{14}{15} = 0.933
 \end{aligned} \tag{1}$$

Then, we compare utilization to the Liu and Layland bound.

$$\begin{aligned}
 LUB &= n \times (2^{1/n} - 1) \\
 &= 3 \times (2^{1/3} - 1) \\
 &= 0.7798 < 0.9333
 \end{aligned} \tag{2}$$

Thus, because the utilization is greater than the LUB, we cannot guarantee that the tasks are schedulable. We will resolve using a timing diagram as seen below.

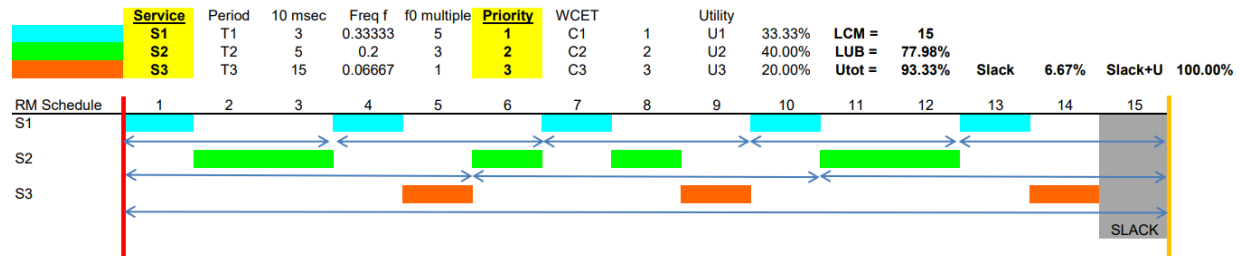


Figure 1.1 Timing Diagram for Rate Monotonic Policy Example

As demonstrated in the timing diagram, all three tasks complete within the hyperperiod. There is even one cycle of idle/slack. The diagram indicates that all deadlines are met for each task and the services are mathematically repeatable. Therefore, the services are feasible.

Although the schedule is mathematically feasible, it is not real-time safe. The CPU utilization is 93.33% and there is only 1ms of idle in the 15ms hyperperiod. According to Liu and Layland's paper, a schedule is real-time safe only if its utilization is less than the least upper bound (LUB). As shown in Equation (2), the utilization (93.33) is greater than the LUB so the schedule is not real-time safe. A worst case scenario such as execution overrun, latency, or context switch overhead could cause S_3 to miss a deadline.

2 Apollo 11 Lunar Lander

Read through the Apollo 11 Lunar lander computer overload story as reported in RTECS Notes, based on this NASA account, and the descriptions of the 1201/1202 events described by chief software engineer Margaret Hamilton as recounted by Dylan Matthews. Summarize the story.

- a) What was the root cause of the overload and why did it violate Rate Monotonic policy?*
- b) Now, read Liu and Layland's paper which describes Rate Monotonic policy and the Least Upper Bound – they derive an equation which advises margin of approximately 30% of the total CPU as the number of services sharing a single CPU core increases.*
- c) Plot this Least Upper bound as a function of number of services.*
- d) Describe three key assumptions Liu and Layland make and document three or more aspects of their fixed priority LUB derivation that you do not understand.*
- e) Would RM analysis have prevented the Apollo 11 1201/1202 errors and potential mission abort? Why or why not?*

2.1 Summary

During the descent to the lunar surface of the Apollo 11 mission, the onboard guidance computer (AGC) threw a 1202 alarm which indicates “executive overload”. The code indicates that the computer has been asked to complete too many tasks. However, due to chief software engineer Margaret Hamilton’s predictive foresight of such a situation, the alarm was able to be dismissed as non-critical, and the landing proceeded safely. The landing was able to complete due to Hamilton’s design that enabled graceful handling of process overload by delaying lower priority tasks.

2.2 Root Cause Analysis

It was later determined that the situation, which was never tested in simulation, was a result of a rendezvous radar switch being left in the “AUTO” mode. This was done as an abort precaution, but the result was the interface pre-empting the CPU, quietly stealing about 13% of the AGC processing time. This violated Rate Monotonic policy because the cycle steals from the radar interface were not accounted for in the utilization calculations. This meant the utilization of periodic tasks exceeded the schedulable bound causing the computer to run out of core sets.

The reason that this did not critically impact the lunar landing was because each time the AGC detected the overload and alarmed, it rebooted, and only restarted critical high-priority tasks, postponing lower-priority tasks. Priority was given to tasks with critical importance. The violation therefore didn’t cause a system crash (or worse spacecraft crash) due to the graceful handling implemented by Hamilton’s team.

2.3 Rate Monotonic Scheduling Analysis

As displayed in Figure 2.1 below, and as described in the paper authored by Liu and Layland, with Rate Monotonic Scheduling, as the number of services, n , grows infinitely large, the Least Upper Bound (LUB) approaches the value, $\ln(2)$.

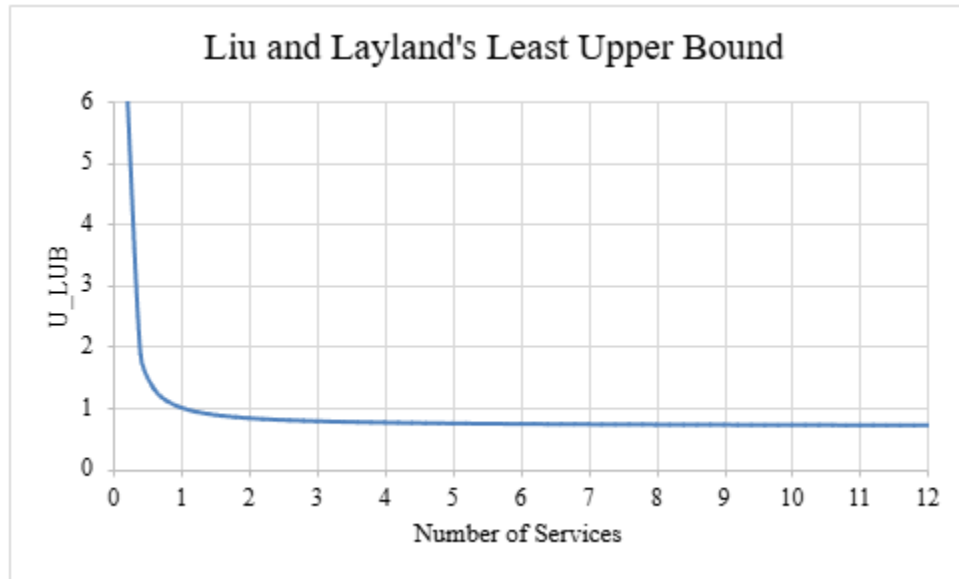


Figure 2.1 Least Upper Bound as a Function of Number of Services

For this LUB approximation, Liu and Leyland make three (3) key assumptions. The first is that each task is strictly periodic with constant hard deadlines equal to one period. The assumption simplifies aperiodic tasks, jitter, and deadlines that may not line up with the period length.

The next assumption is that each task has a known, deterministic, and non-varying worst-case execution time. This completion time is less than the period. This simplifies the derivation to consider the CPU as the only shared resource. It eliminates the effects of caching and synchronization.

The third assumption is that each task is independent of another and can be preempted at any time with no overhead. This reduces the impact of context switching and eliminates shared resources that could cause priority inversion.

The fixed priority LUB derivation by Liu and Leyland is a heavily studied paper. I am having trouble understanding the following aspects. One aspect that I am having understanding is the concept of rate-monotonic priority assignment. It appears to be a bold statement that was not effectively proven in the paper that this type of assignment is globally optimum. The paper does not do a good job of connecting their proof of that statement. Also, I do not fully understand the proof for Theorem 4 as it contains very large logical gaps for the two task derivation. The structure, created by the let statements, is difficult to understand and apply to the following logic. Lastly, as I am having trouble understanding the two task derivation in Theorem 4, the following

Theorem 5 remains difficult to comprehend as well. The author is using the principle of induction to expand the two task derivation from Theorem 4 to a general upper bound. However, it is difficult to track the usage of this proof technique in Theorem 5 and does not explicitly state what it is utilizing from the previous theorem.

2.4 Application to the Apollo 11 Mission

If this paper and theory were available at the time of the Apollo 11 mission design, then the 1201/1202 errors might have been captured before launch. However, this is only true if the engineers applied the policy to all worst-case conditions. As mentioned before, the root cause of the alarms and the potential mission abort was an operational error – the astronauts leaving the rendezvous radar in an active mode despite landing procedures. If this scenario was tested, then engineers would have identified the AGC computer overload. This could have led to future design changes that prevented the scenario entirely. To be clear, simply implementing rate monotonic scheduling would not have corrected the errors during flight. Instead, it probably would have made the situation worse since many high frequency, less critical tasks, including the radar interrupt, were overloading the AGC. With a basic rate monotonic policy implementation, these tasks could be given higher priority due to their shorter period. It would have required significant analysis into that specific scenario and required further design to prevent the computer overload. This shows that theory should not be blindly accepted as best practice.

3 RT-Clock

Download RT-Clock and build it on an R-Pi3b+ or newer and execute the code.

a) Describe what the code is doing and make sure you understand `clock_gettime` and how to use it to time code execution (print or log timestamps between two points in your code).

b) Which clock is best to use? `CLOCK_REALTIME`, `CLOCK_MONOTONIC` or `CLOCK_MONOTONIC_RAW`? Please choose one and update code and improve the commenting.

c) Most RTOS vendors brag about three things: 1) Low Interrupt handler latency, 2) Low Context switch time and 3) Stable timer services where interval timer interrupts, timeouts, and knowledge of relative time has low jitter and drift. Why are each important?

d) Do you believe the accuracy provided by the example RT-Clock code? Why or why not?

3.1 Source Code Description

The RT-Clock source code is performance tool used to measure the precision of the POSIX `nanosleep()` system call using different Linux hardware clocks. The user has the option to use select the hardware clock and whether to use the `SCHED_FIFO` real-time scheduling policy, or stick with the default policy using compile switches as shown in Figure 3.1. If specified, the main thread creates a max priority POSIX thread to run the delay test. Otherwise, the main thread runs the test. The delay test begins by logging the selected clock's resolution. Then, it loops through 100 iterations of the delay test. Each test calls a ten (10) millisecond sleep and

compares that to the *actual* elapsed time using high-precision timestamps. The test calculates the `delay_error` which represents the delta between the desired sleep duration and the actual delay.

```
//#define MY_CLOCK CLOCK_REALTIME
//#define MY_CLOCK CLOCK_MONOTONIC
#define MY_CLOCK CLOCK_MONOTONIC_RAW
//#define MY_CLOCK CLOCK_REALTIME_COARSE
//#define MY_CLOCK CLOCK_MONOTONIC_COARSE

//#define RUN_RT_THREAD
#define TEST_ITERATIONS (100)
```

Figure 3.1 Compile switch to select clock source and scheduling policy

Function breakdown

`main()`

This is the entry point of the application. It handles the thread setup (if applicable) and scheduling policy setup before handing off (or starting) the delay test.

`print_scheduler()`

This is a diagnostic function that identifies which scheduling policy is currently being executed.

`delay_test()`

This function is the core of the program. When ran with a real-time thread, this is the thread function. The function executes the delay test loop, makes the time measurements, and calls for the error calculations.

`delta_t()`

This function computes the math equation for to calculate the delay error. It uses the POSIX timespec struct and prevents underflows by checking the `tv_nsec` and `tv_sec` fields.

`d_ftime()`

This math function transforms the input POSIX timespec struct into readable decimal format.

`End_delay_test()`

This function reports the elapsed time and `delay_error` of each test iteration.

3.2 Which clock is best?

In order to determine which clock is the best to use, I asked Claude AI to create a python program that evaluated the error reported in each test. However, after repeating the executable multiple times and applying the python parsing, I noticed that the averages fluxuated pretty significantly. As you can see in Figure 3.2, CLOCK_REALTIME appeared to have less delay error than CLOCK_MONOTONIC which is very surprising. In order to combat this, I wrote a bash script called run100.sh and asked Claude to modify the python code accordingly. This allowed me to increase my sample size and get more expected results. The bash script runs the executable 100 times, creating 100 test output files, each test of 100 samples. After rebuilding the application with each clock source, and running the bash script for each executable, I created a directory structure that the python file could parse, average, and interpret. This led to the more expected results seen in Figure 3.4. I also prompted to generate a graph for the results. As seen in both Figure 3.3 and Figure 3.5, the delay of the coarse clocks is much greater than the others. The difference between CLOCK_MONOTONIC, CLOCK_MONOTONIC_RAW, and CLOCK_REALTIME was a matter of microseconds.

```
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/RT-Clock$ cat output/plot_rt_clock_delay.out
Loaded 'clock_monotonic_coarse.txt': 100 samples
Loaded 'clock_monotonic_raw.txt': 100 samples
Loaded 'clock_monotonic.txt': 100 samples
Loaded 'clock_realtime_coarse.txt': 100 samples
Loaded 'clock_realtime.txt': 100 samples
```

Rank	File	N	Mean(ns)	Min(ns)	Max(ns)	Std(ns)
1	clock_monotonic_raw.txt	100	11816.3	5024	34720	6227.6
2	clock_realtime.txt	100	12855.5	6161	42929	5550.7
3	clock_monotonic.txt	100	13310.9	5264	38033	6778.9
4	clock_monotonic_coarse.txt	100	1980513.1	0	2000519	199049.1
5	clock_realtime_coarse.txt	100	1980513.1	0	2000519	199049.1

Figure 3.2 Averaging RT-Clock error across 100 samples

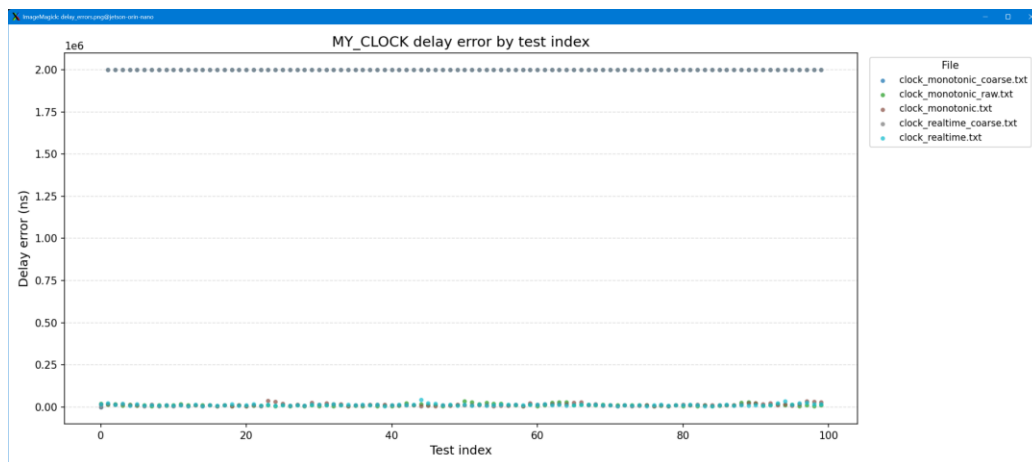


Figure 3.3 Plotting RT-Clock Error across 100 samples

```
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/RT-Clock$ cat output_1/plot_rt_clock_delay_1.out
```

```

Loading 'clock_monotonic' ...
  → 100 trial(s), 100 tests each
Loading 'clock_monotonic_coarse' ...
  → 100 trial(s), 100 tests each
Loading 'clock_monotonic_raw' ...
  → 100 trial(s), 100 tests each
Loading 'clock_realtime' ...
  → 100 trial(s), 100 tests each
Loading 'clock_realtime_coarse' ...
  → 100 trial(s), 100 tests each

```

Rank	Clock	Trials	Mean(ns)	Min(ns)	Max(ns)	Std(ns)
1	clock_monotonic	100	18576.1	12363.6	29698.1	3568.6
2	clock_monotonic_raw	100	20546.2	12064.3	35883.2	4843.6
3	clock_realtime	100	21763.1	13479.0	36578.5	4868.0
4	clock_monotonic_coarse	100	1991220.7	1080226.9	2000423.1	91558.3
5	clock_realtime_coarse	100	1992830.8	1140361.2	2040635.5	85859.2

Figure 3.4 Averaging RT-Clock Error over 100 tests with 100 samples per test

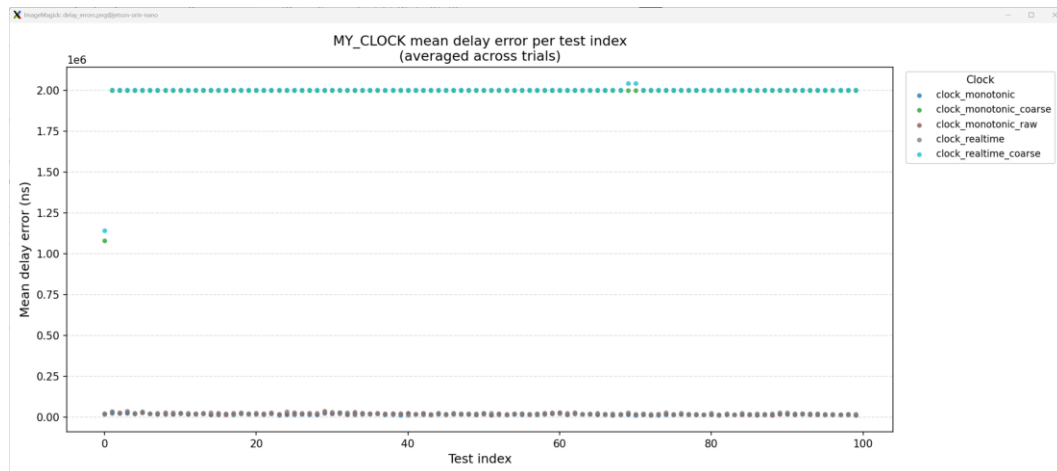


Figure 3.5 Averaging RT-Clock Error over 100 tests with 100 samples per test

Based on the output generated and the analysis done using the python script, we observe that there is significant, greater than 18000ns, average delay error for CLOCK_MONOTONIC. Each other real time clock has greater delay. Also, we not that the delay varies significantly as well. Therefore, the real-time clock is not completely accurate. This could impact critical real-time systems that require exact timing precision.

3.4 Design targets

Low Interrupt Handler Latency

Interrupt latency is the time between hardware interrupt generation and when the processor begins the execution of the Interrupt Service Routine (ISR). In real-time embedded systems, external events often require immediate handling. If there is high latency, the system may not respond fast enough, causing missed deadlines or system

failures. Low interrupt handler latency is important because it minimizes delay between event source and system response, improves determinism with quick ISR times, and allows vendors to guarantee timing requirements for hard deadlines.

Low Context Switch Time

A context switch occurs when the operating system (OS) transfers execution from one task to another. This is done by saving and restoring the processor state and is necessary for multi-tasking and priority-based scheduling. However, it adds overhead during the switch. In real-time systems, reducing that overhead is critical to allow high-priority tasks to begin execution immediately after becoming ready. It also improves the system response to asynchronous events such as interrupts and helps deadlines to be met.

Stable Timer Services

Timers are heavily relied on for periodic tasks, timeouts, measuring time intervals, and synchronizing real-time systems. Low jitter and drift are important characteristics of stable timer services as it directly affects scheduling accuracy.

Jitter: Variation in timing of an event and its expected occurrence.

Drift: Accumulated timing error that causes clock divergence over time.

By reducing jitter and drift, systems can ensure task synchronization, periodic tasks execute as consistent intervals, deterministic system behavior, and timing errors are prevented over longer periods of time.

4 Scheduling using Linux Pthreads

4.1 Simplethread

The simplethread application creates NUM_THREADS (12) POSIX threads each initialized with a parameter based on its index. The index parameter passed, also determines the upper bound of the summation equation that is performed. Each thread calculates the sum of each integer from zero (0) to the thread index. After calculating the sum, the thread logs the sum, and which CPU core it is using before returning.

```

mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/simplethread$ ./pthread
main thread running on CPU=4
Thread idx=0, sum[0...0]=0, running on CPU=2
Thread idx=2, sum[0...2]=3, running on CPU=3
Thread idx=3, sum[0...3]=6, running on CPU=5
Thread idx=4, sum[0...4]=10, running on CPU=3
Thread idx=5, sum[0...5]=15, running on CPU=5
Thread idx=1, sum[0...1]=1, running on CPU=1
Thread idx=6, sum[0...6]=21, running on CPU=3
Thread idx=7, sum[0...7]=28, running on CPU=5
Thread idx=8, sum[0...8]=36, running on CPU=1
Thread idx=9, sum[0...9]=45, running on CPU=3
Thread idx=10, sum[0...10]=55, running on CPU=3
Thread idx=11, sum[0...11]=66, running on CPU=3
TEST COMPLETE
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/simplethread$ ./pthread
main thread running on CPU=2
Thread idx=0, sum[0...0]=0, running on CPU=0
Thread idx=1, sum[0...1]=1, running on CPU=4
Thread idx=2, sum[0...2]=3, running on CPU=1
Thread idx=3, sum[0...3]=6, running on CPU=0
Thread idx=4, sum[0...4]=10, running on CPU=5
Thread idx=5, sum[0...5]=15, running on CPU=0
Thread idx=6, sum[0...6]=21, running on CPU=5
Thread idx=7, sum[0...7]=28, running on CPU=0
Thread idx=8, sum[0...8]=36, running on CPU=5
Thread idx=9, sum[0...9]=45, running on CPU=5
Thread idx=10, sum[0...10]=55, running on CPU=5
Thread idx=11, sum[0...11]=66, running on CPU=5
TEST COMPLETE

```

Figure 4.1 Simplethread application output

The simplethread-affinity application focuses on CPU affinity and thread scheduling. The application tests NUM_THREADS (64) high priority counter threads, which are all restricted to CPU 3 by using the affinity POSIX API.

First, the main thread (entry point) first calls the function `set_scheduler()` which establishes the scheduling policy, thread priority, and CPU pinning for the application threads. Then, it logs the affinity mask of the main thread and which core it is running on. Finally it creates the starter thread.

The function, `set_scheduler()`, sets the scheduling policy to `SCHED_FIFO` and enforces CPU pinning to core three (3) using the `pthread_attr_setaffinity_np()` API. This enforces controller thread and counter threads to all run on CPU core 3. It also assigns the maximum priority to the thread attributes.

The starterthread, created by the main thread, executes its thread function, `starterThread()`. This function logs which core it is running on, always core 3 due to CPU affinity, and then runs a loop to spin up the defined number of counter threads. Each counter thread is created with its index (0-63) as a parameter. The starterthread then waits for all of the counter threads to finish using `pthread_join()`;

The counter threads execute the thread function, `counterThread()`. Each thread records a timestamp to the start of the thread execution. Then, it begins doing work, summing the integers from zero to its thread index `MAX_ITERATIONS` (1,000,000) times. Once the work is done, the sum and another timestamp are recorded to indicate the stop of the thread execution.

The output described above can be seen below in Figure 4.2

```

INITIAL Pthread policy is SCHED_OTHER
ADJUSTED Pthread policy is SCHED_FIFO
main thread running on CPU=2, CPUs = 0 1 2 3 4 5
starter thread running on CPU=3

Thread idx=0, sum[0...0]=0, running on CPU=3, start=1780587929.986866, stop=1780587929.990918
Thread idx=1, sum[0...1]=1, running on CPU=3, start=1780587929.991112, stop=1780587929.995549
Thread idx=2, sum[0...2]=3, running on CPU=3, start=1780587929.995594, stop=1780587930.002403
Thread idx=3, sum[0...3]=6, running on CPU=3, start=1780587930.002444, stop=1780587930.011823
Thread idx=4, sum[0...4]=10, running on CPU=3, start=1780587930.011866, stop=1780587930.024379
Thread idx=5, sum[0...5]=15, running on CPU=3, start=1780587930.024421, stop=1780587930.040053
Thread idx=6, sum[0...6]=21, running on CPU=3, start=1780587930.040097, stop=1780587930.059147
Thread idx=7, sum[0...7]=28, running on CPU=3, start=1780587930.059190, stop=1780587930.081887
Thread idx=8, sum[0...8]=36, running on CPU=3, start=1780587930.081921, stop=1780587930.108493
Thread idx=9, sum[0...9]=45, running on CPU=3, start=1780587930.108644, stop=1780587930.138717
Thread idx=10, sum[0...10]=55, running on CPU=3, start=1780587930.138758, stop=1780587930.174045
Thread idx=11, sum[0...11]=66, running on CPU=3, start=1780587930.174090, stop=1780587930.213624
Thread idx=12, sum[0...12]=78, running on CPU=3, start=1780587930.213669, stop=1780587930.259922
Thread idx=13, sum[0...13]=91, running on CPU=3, start=1780587930.259965, stop=1780587930.310521
Thread idx=14, sum[0...14]=105, running on CPU=3, start=1780587930.310561, stop=1780587930.362756
Thread idx=15, sum[0...15]=120, running on CPU=3, start=1780587930.362799, stop=1780587930.419296
Thread idx=16, sum[0...16]=136, running on CPU=3, start=1780587930.419337, stop=1780587930.480359
Thread idx=17, sum[0...17]=153, running on CPU=3, start=1780587930.480398, stop=1780587930.545856
Thread idx=18, sum[0...18]=171, running on CPU=3, start=1780587930.545897, stop=1780587930.615919
Thread idx=19, sum[0...19]=190, running on CPU=3, start=1780587930.615958, stop=1780587930.690412
Thread idx=20, sum[0...20]=210, running on CPU=3, start=1780587930.690450, stop=1780587930.769401
Thread idx=21, sum[0...21]=231, running on CPU=3, start=1780587930.769441, stop=1780587930.852613
Thread idx=22, sum[0...22]=253, running on CPU=3, start=1780587930.852652, stop=1780587930.985711
Thread idx=23, sum[0...23]=276, running on CPU=3, start=1780587930.985907, stop=1780587931.078471
Thread idx=24, sum[0...24]=300, running on CPU=3, start=1780587931.078524, stop=1780587931.175404
Thread idx=25, sum[0...25]=325, running on CPU=3, start=1780587931.175454, stop=1780587931.276777
Thread idx=26, sum[0...26]=351, running on CPU=3, start=1780587931.276820, stop=1780587931.383207
Thread idx=27, sum[0...27]=378, running on CPU=3, start=1780587931.383248, stop=1780587931.494180
Thread idx=28, sum[0...28]=406, running on CPU=3, start=1780587931.494222, stop=1780587931.611892
Thread idx=29, sum[0...29]=435, running on CPU=3, start=1780587931.611938, stop=1780587931.734078
Thread idx=30, sum[0...30]=465, running on CPU=3, start=1780587931.734124, stop=1780587931.857958
Thread idx=31, sum[0...31]=496, running on CPU=3, start=1780587931.857998, stop=1780587932.040471
Thread idx=32, sum[0...32]=528, running on CPU=3, start=1780587932.040556, stop=1780587932.173304
Thread idx=33, sum[0...33]=561, running on CPU=3, start=1780587932.173349, stop=1780587932.310541
Thread idx=34, sum[0...34]=595, running on CPU=3, start=1780587932.310589, stop=1780587932.452292
Thread idx=35, sum[0...35]=630, running on CPU=3, start=1780587932.452333, stop=1780587932.598495
Thread idx=36, sum[0...36]=666, running on CPU=3, start=1780587932.598541, stop=1780587932.749224
Thread idx=37, sum[0...37]=703, running on CPU=3, start=1780587932.749265, stop=1780587932.904413
Thread idx=38, sum[0...38]=741, running on CPU=3, start=1780587932.904453, stop=1780587933.112641
Thread idx=39, sum[0...39]=780, running on CPU=3, start=1780587933.112745, stop=1780587933.276808
Thread idx=40, sum[0...40]=820, running on CPU=3, start=1780587933.276851, stop=1780587933.445448
Thread idx=41, sum[0...41]=861, running on CPU=3, start=1780587933.445629, stop=1780587933.618748
Thread idx=42, sum[0...42]=903, running on CPU=3, start=1780587933.618803, stop=1780587933.797134
Thread idx=43, sum[0...43]=946, running on CPU=3, start=1780587933.797183, stop=1780587934.028457
Thread idx=44, sum[0...44]=990, running on CPU=3, start=1780587934.028547, stop=1780587934.218228
Thread idx=45, sum[0...45]=1035, running on CPU=3, start=1780587934.218277, stop=1780587934.412519
Thread idx=46, sum[0...46]=1081, running on CPU=3, start=1780587934.412565, stop=1780587934.608188
Thread idx=47, sum[0...47]=1128, running on CPU=3, start=1780587934.608233, stop=1780587934.808320
Thread idx=48, sum[0...48]=1176, running on CPU=3, start=1780587934.808363, stop=1780587935.061185
Thread idx=49, sum[0...49]=1225, running on CPU=3, start=1780587935.061286, stop=1780587935.270277
Thread idx=50, sum[0...50]=1275, running on CPU=3, start=1780587935.270326, stop=1780587935.483781
Thread idx=51, sum[0...51]=1326, running on CPU=3, start=1780587935.483828, stop=1780587935.701857
Thread idx=52, sum[0...52]=1378, running on CPU=3, start=1780587935.701910, stop=1780587935.924324
Thread idx=53, sum[0...53]=1431, running on CPU=3, start=1780587935.924372, stop=1780587936.199842
Thread idx=54, sum[0...54]=1485, running on CPU=3, start=1780587936.199957, stop=1780587936.431370
Thread idx=55, sum[0...55]=1540, running on CPU=3, start=1780587936.431422, stop=1780587936.667317
Thread idx=56, sum[0...56]=1596, running on CPU=3, start=1780587936.667361, stop=1780587936.907700
Thread idx=57, sum[0...57]=1653, running on CPU=3, start=1780587936.907745, stop=1780587937.202461
Thread idx=58, sum[0...58]=1711, running on CPU=3, start=1780587937.202583, stop=1780587937.452777
Thread idx=59, sum[0...59]=1770, running on CPU=3, start=1780587937.452828, stop=1780587937.707490
Thread idx=60, sum[0...60]=1830, running on CPU=3, start=1780587937.707540, stop=1780587938.018866
Thread idx=61, sum[0...61]=1891, running on CPU=3, start=1780587938.018971, stop=1780587938.285161
Thread idx=62, sum[0...62]=1953, running on CPU=3, start=1780587938.285216, stop=1780587938.552564
Thread idx=63, sum[0...63]=2016, running on CPU=3, start=1780587938.552611, stop=1780587938.824396
TEST COMPLETE

```

FIGURE 4.2 Simplethread-affinity application output

Simplethread-affinity differs from simplethread primarily because it enforces CPU pinning to one core using the POSIX CPU affinity. This causes the execution to be done serially, since only one thread can be ran on the core at once. The simplethread-affinity application also demonstrates the SCHED_FIFO scheduling policy as it handles the first in threads first. This type of scheduling causes the threads to be run serially, since they are equal priority. This differs from other policies such as SCHED_RR and SCHED_OTHER.

4.2 Incdecthread

```
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/incdecthread$ ./pthread
Main thread gsum=4967, agsum=0
TEST COMPLETE
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/incdecthread$ ./pthread
Main thread gsum=-29506, agsum=0
TEST COMPLETE
```

Figure 4.3 Application output of incdecthread demonstrating race condition

This application demonstrates race conditions and the importance of atomic accesses for shared memory. The application creates two threads, one to increment and one to decrement a global counter one million (1,000,000) times each. The application uses two sum counters, *gsum*, which is a standard int, and *agsum*, which is a POSIX *atomic_int*. The application shows the danger of having multiple threads modify unprotected shared data (*gsum*) and how to protect it, using atomic access. We observe that *gsum* varies in the two executions of the application in Figure XX above. This shows that the race condition we see, a read-modify-write, is not deterministic. However, the atomic protected shared memory is deterministic and accurate.

POSIX API Calls

- `pthread_t`
 - This is the thread object type. In the case of this file, two threads are created with default parameter and the only parameter is the thread ID.
- `pthread_create()`
 - This API is used to create a thread with the specified descriptor (pointer), attributes, entry point (function), and parameters (thread ID)
- `pthread_join()`
 - This API allows the main thread to wait for all other threads to complete before proceeding.

C11 atomic types and operations

- `atomic_int`
 - This is the type for an atomic integer. The type allows atomic operations on the variable.

- `atomic_fetch_add()`
 - This API is used to perform atomic addition. It requires an atomic variable type to modify it. The function returns the value of the variable object held prior to the operation.
- `atomic_fetch_sub()`
 - This API is used to perform atomic subtraction. It requires an atomic variable type to modify it. The function returns the value of the variable object held prior to the operation.

Reference: `testdigest` – Reference for how semaphores are used to synchronize threads.

Changes made to get working correctly:

- `testdigest.c`
 - Add header files `unistd.h` – for `getpid()` – and `string.h` – for `strlen()`.
 - To compare scheduling policies, defined the compile switch `SET_SCHED_POLICY`
 - Deleted the final character in `test` as it was prompting a `strlen()` warning that there was no null terminator
- `crc.c`
 - added casting to `crc` type to prevent overflow and underflow warnings

Synchronizing `incdecthread` using semaphores

Initialize the semaphore to be shared between both the increment and decrement threads and to be initially available. Before modifying the unprotected `gsum` data, the threads must acquire the semaphore using `sem_wait()`, after modifying the data, the thread releases the semaphore using `sem_post()`. After both threads are complete and joined, the semaphore is safely destroyed.

The use of the semaphore makes the critical section safe, and the program appears to be deterministic. However, I did notice that the execution time increased due to the overhead of waiting, acquiring, and releasing the semaphore.

```
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/incdecthread$ sudo ./pthread_sem
Main thread gsum=0, agsum=0
TEST COMPLETE
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/incdecthread$ sudo ./pthread_sem
Main thread gsum=0, agsum=0
TEST COMPLETE
```

Figure 4.4 Introducing semaphores to prevent `gsum` race condition

Synchronizing `incdecthread` using `SCHED_FIFO`

Before creating each of the threads, initialize `pthread` attributes. Set the thread attributes to have the `SCHED_FIFO` scheduling policy and we must use affinity to ensure that the threads run on

the same core to promote determinism. Also, create parameters for the threads with max priorities. Then, create the threads with the set attributes and parameters. This will cause the increment thread to finish first, and the decrement thread to finish second, effectively creating a series run because we are running using SCHED_FIFO and limiting the execution of both the increment and decrement to one core.

```
● mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/incdecthread$ sudo ./pthread_fifo
Main thread running on CPU 4
Inc thread running on CPU 3
Dec thread running on CPU 3
Main thread gsum=0, agsum=0
TEST COMPLETE
● mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/incdecthread$ sudo ./pthread_fifo
Main thread running on CPU 0
Inc thread running on CPU 3
Dec thread running on CPU 3
Main thread gsum=0, agsum=0
TEST COMPLETE
```

Figure 4.5 Introducing CPU affinity to prevent gsum race condition

Comparison

The application using SCHED_FIFO is over twenty (20) times faster than the solution using semaphores. It is also slightly faster than the original solution. See Figure 4.6. Despite only using one core, the execution time of incdecthread using SCHED_FIFO policy is much faster due to the reduced context switching and semaphore waiting. In a more complex application, it is advised to utilize both semaphores and the SCHED_FIFO scheduling policy to effectively

```

mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/incdecthread$ time sudo ./pthread
Main thread gsum=2558, agsum=0
TEST COMPLETE

real    0m0.656s
user    0m0.011s
sys     0m0.021s
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/incdecthread$ time sudo ./pthread_sem
Main thread gsum=0, agsum=0
TEST COMPLETE

real    0m1.855s
user    0m0.016s
sys     0m0.015s
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaffin/incdecthread$ time sudo ./pthread_fifo
Main thread running on CPU 5
Inc thread running on CPU 3
Dec thread running on CPU 3
Main thread gsum=0, agsum=0
TEST COMPLETE

real    0m0.069s
user    0m0.021s
sys     0m0.010s

```

Figure 4.6 Time comparison of incdecthread with synchronization techniques

4.3 Deadline – SCHED_DEADLINE and SCHED_FIFO

The text output is of the following format – “[CLOCK_REALTIME] – Thread A running for [elapsed_ns] ns. The *elapsed_ns* variable is derived using CLOCK_MONOTONIC compared to the first entry into the thread function.

The first thing noticed with the SCHED_DEADLINE policy was that it did not have any effect unless the application was run with superuser privileges. The largest impact of using SCHED_DEADLINE, as evidenced by the reduced output, is that it treats the thread as a periodic task. Using the sched_yield() API, the thread notifies the kernel that the allocated runtime (10ms) is complete, so the thread should yield and sleep until the period is up, and the next runtime execution may begin. In our example, we added a check to see how long the *Thread A* has been running. We also, added a check that if the time elapsed from when Thread A first ran has exceeded 6 seconds, then the application will stop. This was done to compare the outputs more accurately. As seen in Figure 4.7 below, last line of output is after the 4th cycle of running Thread A – note the check for the time elapsed happens sequentially after the print statement.

```

1  [1780626068.376182798] - Thread A running on CPU 2
2  [1780626068.376399380] - Thread A running for 216806 ns
3  [1780626070.376393705] - Thread A running for 2000211259 ns
4  [1780626072.376479678] - Thread A running for 4000297168 ns
5  [1780626074.376524303] - Thread A running for 6000341825 ns

```

Figure 4.7 Deadline using SCHED_DEADLINE scheduling policy

When using SCHED_FIFO, the deadline parameters (sched_runtime, sched_period, sched_deadline) are ignored by the kernel. The kernel only considers the threads priority for scheduling tasks. When the thread calls sched_yield(), the thread is put in the back of the queue of threads to be run. Since there is only one thread in this application, it runs immediately, producing more text output. As seen in Figure 4.8, the application produces over 100,000 lines of output in the 6 seconds that it was run. Again, the elapsed runtime check terminates the program.

```

104080 [1780627250.524972789] - Thread A running for 5958015888 ns
104081 [1780627250.524975477] - Thread A running for 5958018576 ns
104082 [1780627250.524978165] - Thread A running for 5958021296 ns
104083 [1780627250.524980917] - Thread A running for 5958023984 ns
104084 [1780627250.581894326] - Thread A running for 6014937553 ns

```

Figure 4.8 Deadline using SCHED_FIFO scheduling policy

4.4 Sequencer

I originally tried to implement this task without a sequencer thread and just use SCHED_DEADLINE. However, after endless debugging, and attempts at very hacky linux configurations, I ended my efforts. I attempted multiple ways to constrain the threads to one core and use SCHED_DEADLINE. I attempted to use the pthread affinity APIs, cgroups, and running the application as a system service. I also tried to use the isolcpus kernel boot parameter and taskset with no success. All of these created an “Operation not permitted” error on the sched_attr syscall.

The reason that SCHED_DEADLINE is unsuitable for this problem is that it does not accept affinity. It was not possible to pin a thread to a specific CPU core and configure the SCHED_DEADLINE parameters (runtime, period, deadline). The application worked on multiple threads, busy waiting for the entire specified runtime, but there was no sequencing possible. In order to implement the sequencing, a switch to SCHED_FIFO and a sequencer thread was necessary.

In order to implement the solution, a sequencer thread with max priority is created. It releases the initial semaphores to release F10 and F20 and then sleeps for 10ms before waking. Upon wake, it determines if it is time to release F10 or F20 and then returns to sleep. The sequencer thread is

created last to allow the service threads to a chance to be ready for the semaphores as soon as they are released.

The F10 service has runtime of ten (10) milliseconds and deadline and period of twenty (20) milliseconds. The F20 service has a runtime of twenty (20) milliseconds and deadline and period of fifty (50) milliseconds. Reference Figure 4.9 below. The total utilization is 90% as derived from Equation (3). The hyperperiod is the least common multiple (LCM) of the services' periods – LCM(20,50) = 100ms.

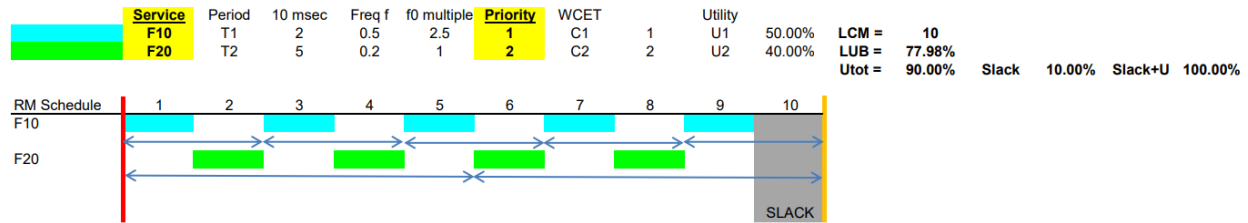


Figure 4.9 Timing Diagram for Sequencer Problem

$$\begin{aligned}
 U &= \sum_{i=1}^n \frac{C_i}{T_i} \\
 &= \frac{10}{20} + \frac{20}{50} \\
 &= \frac{90}{100} = 0.9
 \end{aligned} \tag{3}$$

The F10 service has a priority slightly less than the scheduler but higher than the F20 task because it has a shorter period. These services wait for their related semaphore controlled by the sequencer to be released, upon which they log their release, busy wait for their previously defined runtime, and then log their completion. The release and completion logs use CLOCK_MONOTONIC to show the elapsed time that has passed for each release and completion event. The busy wait function uses CLOCK_THREAD_CPUTIME_ID as opposed to CLOCK_MONOTONIC. This is to ensure that the thread runs for the allocated runtime even in the case of preemption. By using this clock, the calling threads processor time is used as opposed to wall-clock time. This allows service F20 to be preempted and then still busy wait for its defined runtime.

The application uses syslog to log all the release and completion events as shown in Figure 4.10. The application dynamically pins all the threads created (Sequencer, F10, and F20) to the CPU core that the main application runs on. This is to demonstrate the synchronization of the

sequencer application. The application implements signal handling to safely handle SIGINT, SIGTERM, and SIGALARM. The main thread creates a timer using the POSIX alarm() API that the user can modify by changing the APP_TIME_SEC macro. This sets the duration of the application, unless it is interrupted by another signal which the user can trigger using ‘CTRL+C’.

```
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 started on CPU 4
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F20 started on CPU 4
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: SEQ started on CPU 4
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 1 - RELEASE @ 1.539 ms
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 1 - COMPLETE @ 11.457 ms (complete 9.92 after release)
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F20 - JOB 1 - RELEASE @ 11.501 ms
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 2 - RELEASE @ 21.488 ms
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 2 - COMPLETE @ 31.311 ms (complete 9.82 after release)
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F20 - JOB 1 - COMPLETE @ 41.181 ms (complete 29.68 after release)
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 3 - RELEASE @ 41.490 ms
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 3 - COMPLETE @ 51.325 ms (complete 9.84 after release)
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F20 - JOB 2 - RELEASE @ 51.515 ms
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 4 - RELEASE @ 61.491 ms
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 4 - COMPLETE @ 71.315 ms (complete 9.82 after release)
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F20 - JOB 2 - COMPLETE @ 81.226 ms (complete 29.71 after release)
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 5 - RELEASE @ 81.487 ms
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 5 - COMPLETE @ 91.321 ms (complete 9.83 after release)
Jun  5 12:36:03 jetson-orin-nano ex1_q4d[289649]: F10 - JOB 6 - RELEASE @ 101.490 ms
```

Figure 4.10 Syslog output for sequencer application

As seen in the figure above, the F10 and F20 service threads closely follow the expected execution timeline. The services do not miss a deadline and there is idle time for the last ten milliseconds (10ms) of the LCM. Although the trace closely matches the original VxWorks schedule, small deviations are expected due to Linux scheduler latency, timer resolution, system-call overhead, and logging operations as Linux is not a hard real-time operating system.

5 References

- [1] J. Pratt and S. Siewert, Real-Time Embedded Components and Systems with Linux and RTOS, 2nd ed. Herndon, VA, USA: Mercury Learning and Information, 2016.
- [2] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," J. ACM, vol. 20, no. 1, pp. 46–61, Jan. 1973.
- [3] J. D. Powell, J. A. White, and G. Brand, "Advanced digital autopilot for transport aircraft," National Aeronautics and Space Administration, Washington, DC, USA, NASA Tech. Memo. TM X-73093, Jan. 1976. Available: <https://ntrs.nasa.gov/citations/19760005868>
- [4] NASA, "The Apollo Lunar Surface Journal and The Apollo Flight Journal." NASA History Division. Accessed: Jun. 5, 2026. [Online]. Available: <https://www.nasa.gov/history/alsj-and-afj/>
- [5] E. M. Jones and K. Glover, "Apollo 11 Landing," Apollo Lunar Surface Journal. Accessed: Jun. 5, 2026. [Online]. Available: <https://apollojournals.org/alsj/a11/a11.landing.html>
- [6] Desmos, "Desmos Graphing Calculator." Desmos. Accessed: Jun. 5, 2026. [Online]. Available: <https://www.desmos.com/calculator>
- [7] "Linux man pages," Die.net. Accessed: Jun. 5, 2026. [Online]. Available: <https://linux.die.net/man/>
- [8] M. Kerrisk, "Linux man-pages online," Man7.org. Accessed: Jun. 5, 2026. [Online]. Available: <http://man7.org/linux/man-pages/index.html>

6 Appendix

Github repository: https://github.com/mamc3334/RTES_5623

Pull request: https://github.com/mamc3334/RTES_5623/pull/1/changes

Forked RTES Github repository: <https://github.com/mamc3334/RTES-ECEE-5623-mcgaffin>

Pull request: <https://github.com/mamc3334/RTES-ECEE-5623-mcgaffin/pull/1/changes>